

CaptionAI: AI-Powered Debt Relief & Financial Recovery Platform

Project Description:

The AI Powered Debt Relief & Financial Recovery Platform is an intelligent web-based financial assistance system developed to simplify and automate the debt management and financial recovery process for borrowers. The platform enables users to securely manage their loan information, analyze their financial health, receive AI-powered settlement recommendations, and generate intelligent negotiation strategies for communicating with lenders.

The application integrates Artificial Intelligence, financial analytics, and modern web technologies to provide borrowers with personalized financial guidance. By analyzing borrower income, monthly expenses, outstanding loans, overdue duration, and repayment capability, the system calculates important financial metrics such as EMI-to-Income Ratio, Debt-to-Income Ratio, Monthly Surplus, Debt Stress Level, Financial Health Score, and Settlement Percentage. These insights help borrowers understand their financial condition and make informed repayment decisions.

The platform also incorporates Google Gemini AI to generate personalized lender-specific negotiation strategies and professional settlement request letters. When the AI service is unavailable, a rule-based fallback mechanism ensures uninterrupted platform functionality and stable financial recommendations.

Developed using React.js for the frontend and FastAPI with Python for the backend, the application follows a modular full-stack architecture that ensures scalability, maintainability, and secure communication between frontend and backend components. SQLite and SQLAlchemy ORM are used for efficient data storage and management, while JWT Authentication provides secure user authentication and session management.

The Financial Dashboard offers borrowers a comprehensive overview of their financial status through interactive metrics, settlement analysis, loan prioritization, and AI-generated recommendations. The platform also maintains historical AI negotiation records and financial data, allowing users to monitor their financial recovery progress over time.

Scenario 1: AI-Powered Settlement Recommendation

A borrower adds loan details including outstanding amount, EMI, overdue duration, and monthly income. The system analyzes the financial profile and generates personalized settlement recommendations along with debt stress analysis and financial health insights.

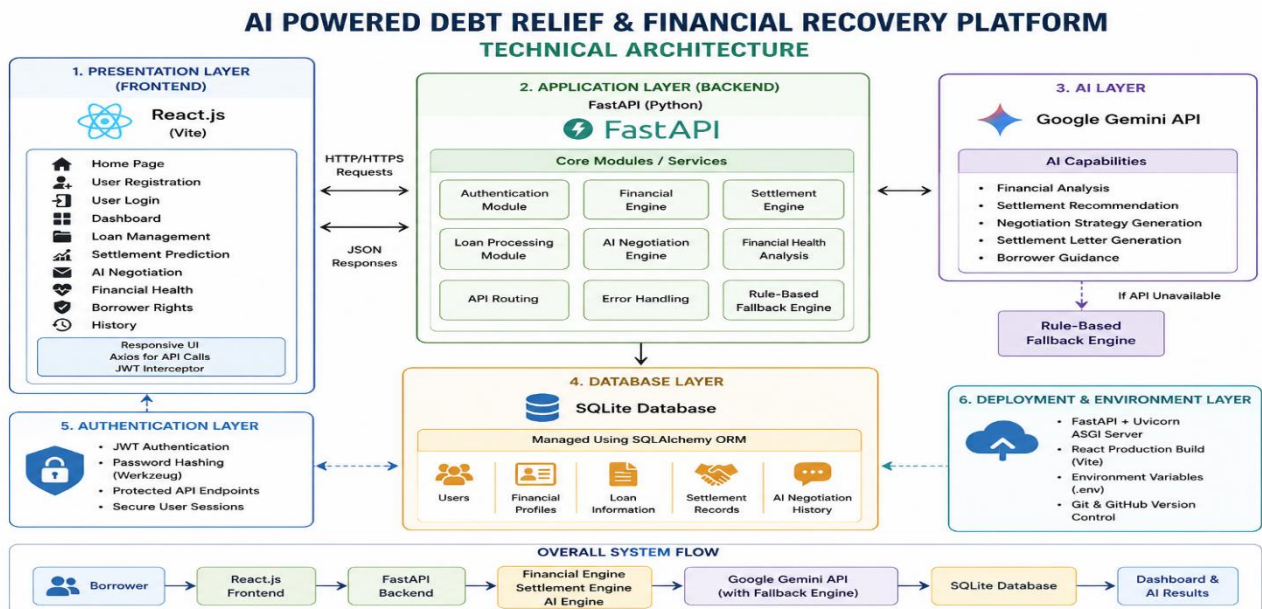
Scenario 2: Intelligent Negotiation Letter Generation

A borrower selects a loan account and requests a negotiation strategy. The platform uses Google Gemini AI to generate professional lender-specific negotiation emails and settlement request letters based on the borrower's financial condition and repayment capability.

Scenario 3: Financial Health Tracking & Loan Analysis

A borrower accesses the dashboard to monitor financial health metrics such as monthly surplus, EMI ratio, debt stress level, and settlement percentages. The platform also maintains AI-generated negotiation history and provides borrower guidance for improving debt management decisions.

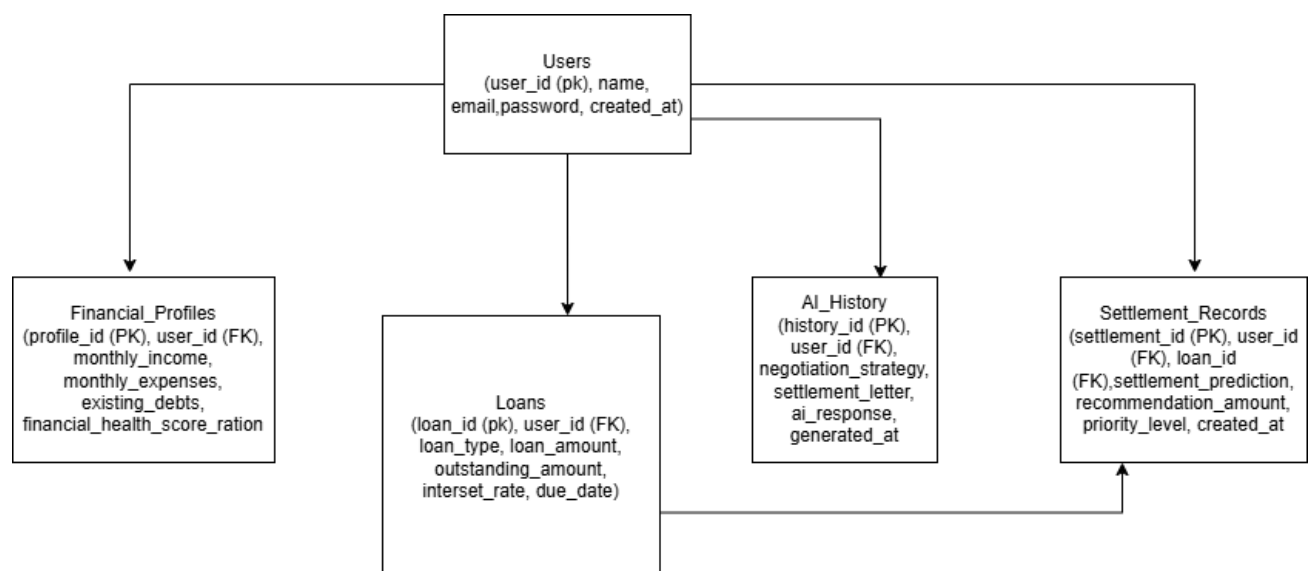
Technical Architecture:



Description:

The AI Powered Debt Relief & Financial Recovery Platform follows a modular full-stack architecture consisting of a React.js (Vite) frontend, a FastAPI (Python) backend, Google Gemini AI integration, and an SQLite database managed with SQLAlchemy ORM. The frontend communicates securely with the backend through REST APIs, while JWT authentication ensures secure user access. The backend processes financial analysis, settlement prediction, and AI-powered negotiation strategies, with a rule-based fallback mechanism for reliability. This architecture provides a secure, scalable, and efficient platform for intelligent debt management and financial recovery.

ER Diagram :



Description:

The ER Diagram represents the database structure of the FinRelief AI – AI Powered Debt Relief & Financial Recovery Platform. It consists of five main entities: Users, Loans, Financial_Profiles, Settlement_Records, and AI_History. The Users table acts as the central entity, where each borrower can have multiple loans, settlement records, and AI-generated negotiation histories, while each borrower has a single financial profile. Foreign key relationships maintain data integrity between tables, enabling secure storage of borrower information, loan details, financial analysis, AI-generated negotiation strategies, and settlement recommendations. This normalized database design minimizes redundancy, improves data consistency, and supports efficient financial processing, AI integration, and future scalability of the platform.

Pre-requisites:

- **Python 3.11+:** <https://www.python.org/downloads/>
- **FastAPI :** <https://fastapi.tiangolo.com/>
- **React.js :** <https://react.dev/>
- **Vite :** <https://vitejs.dev/>
- **Google Gemini API:** <https://aistudio.google.com/app/apikey>
- **SQLAlchemy ORM:** <https://www.sqlalchemy.org/>
- **SQLite :** <https://www.sqlite.org/docs.html>
- **Git Version Control:** <https://git-scm.com/doc>
- **GitHub Repository:** <https://github.com/>
- **Python Virtual Environment (venv):** <https://docs.python.org/3/library/venv.html>
- **Visual Studio Code:** <https://code.visualstudio.com/>
- **JWT Authentication :** <https://jwt.io/>

Project Workflow:

Epic 1. FinRelief AI Application Development & System Setup

- Story 1: Python Backend Environment Setup
- Story 2: Backend Dependency Installation Using Requirements.txt
- Story 3: Frontend Setup (React.js + Vite)
- Story 4: Project Structure & Full-Stack Directory Organization

Epic 2. AI Integration & Financial Processing Setup

- Story 1: FastAPI Backend & API Endpoints
- Story 2: Financial Engine Module
- Story 3: Settlement Prediction System
- Story 4: AI Negotiation Strategy Engine
- Story 5: Fallback Logic Implementation

Epic 3. Database Management & Financial Data Storage Setup

- Story 1: API Development Functionality
- Story 2: Loan & Settlement Processing Functionality
- Story 3: Data Handling

Epic 4. Frontend Integration & User Interface Development

- Story 1: User Interface Development
- Story 2: Frontend Communication With FastAPI
- Story 3: Financial Health Metrics & Settlement Data Visualization
- Story 4: UI Enhancements

Epic 5. Testing, Debugging & Performance Optimization

- Story 1: System Testing
- Story 2: Backend Error Handling & AI Fallback Management
- Story 3: Performance Optimization & Secure Session Management

Epic 6. Version Control, Project Finalization & Deployment Readiness

- Story 1: GitHub Repository Setup & Version Control Management
- Story 2: Project Cleanup & Modular Folder Structure Organization
- Story 3: Deployment Configuration & Production Readiness Setup

Epic 7. Conclusion

Epic 1. FinRelief AI Application Development & System Setup

It Focuses on establishing the complete development environment and project foundation for the AI Powered Debt Relief & Financial Recovery Platform (FinRelief AI). During this phase, the backend and frontend environments were configured, project dependencies were installed, and a modular full-stack project structure was created. This setup provides a stable foundation for implementing AI-powered financial analysis, loan management, settlement prediction, and negotiation features in later epics.

Story 1: Python Backend Environment Setup

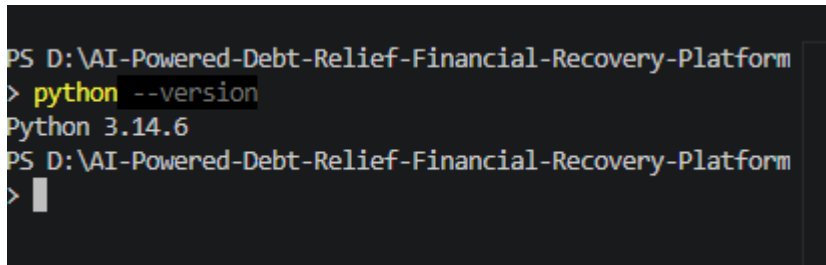
The backend development environment was configured using Python 3.11. A dedicated virtual environment was created to isolate project dependencies and ensure consistent execution across different development systems.

Steps Performed:

Step 1: Installed Python 3.11.

Verified Python installation using:

`python --version`



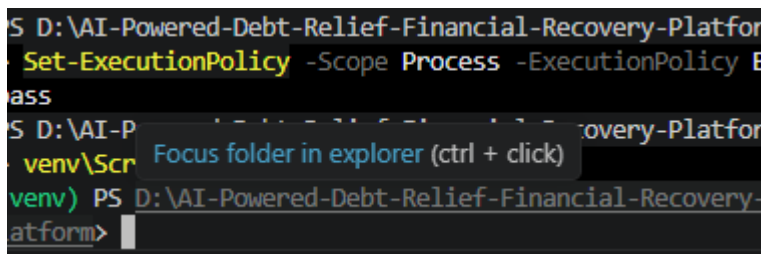
```
PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform
> python --version
Python 3.14.6
PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform
>
```

Step 2: Created a virtual environment:

`python -m venv venv`

Step 3: Activated the virtual environment:

`venv\Scripts\activate`



```
PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform
> Set-ExecutionPolicy -Scope Process -ExecutionPolicy B
ass
PS D:\AI-P...overy-Platform
> venv\Scr Focus folder in explorer (ctrl + click)
venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-
atform>
```

Verified successful activation before installing dependencies.

Story 2: Backend Dependency Installation

All backend libraries required for API development, database connectivity, authentication, AI integration, and financial processing were installed using the requirements.txt file.

Steps Performed

Created requirements.txt. Installed dependencies using:

`pip install -r requirements.txt`

```
(venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\backend> pip install -r requirements.txt
Requirement already satisfied: fastapi in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from -r requirements.txt (line 1)) (0.138.0)
Requirement already satisfied: uvicorn in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from -r requirements.txt (line 2)) (0.49.0)
Requirement already satisfied: sqlalchemy in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from -r requirements.txt (line 3)) (2.0.30)
Requirement already satisfied: python-dotenv in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from -r requirements.txt (line 4)) (0.21.0)
Requirement already satisfied: python-multipart in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from -r requirements.txt (line 5)) (0.0.9)
Requirement already satisfied: ecdsa!=0.15.* in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from -r requirements.txt (line 6)) (0.17.0)
Requirement already satisfied: annotated-types<0.6.0 in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from pydantic>=2.9.0->fastapi->-r requirements.txt (line 1)) (0.7.0)
Requirement already satisfied: pydantic-core==2.46.4 in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from pydantic>=2.9.0->fastapi->-r requirements.txt (line 1)) (2.46.4)
Requirement already satisfied: anyio<5,>=3.6.2 in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from starlette>=0.46.0->fastapi->-r requirements.txt (line 1)) (4.14.1)
Requirement already satisfied: httplib2<1.0.0,>=0.19.0 in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from google-api-python-client->google-generativeai->-r requirements.txt (line 8)) (0.32.0)
Requirement already satisfied: google-auth-httplib2<1.0.0,>=0.2.0 in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from google-api-python-client->google-generativeai->-r requirements.txt (line 8)) (0.4.0)
Requirement already satisfied: uritemplate<5,>=3.0.1 in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from google-api-python-client->google-generativeai->-r requirements.txt (line 8)) (4.2.0)
Requirement already satisfied: pyparsing<4,>=3.1 in D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\venv\Lib\site-packages (from httplib2<1.0.0,>=0.19.0->google-api-python-client->google-generativeai->-r requirements.txt (line 8)) (3.3.2)
(venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\backend>
```


Installed and verified the following packages:

- FastAPI
- Uvicorn
- SQLAlchemy
- python-dotenv
- google-generativeai
- python-jose
- Werkzeug
- psycpg2-binary

Verified successful installation without dependency conflicts.

Story 3: Frontend Setup (React.js + Vite)

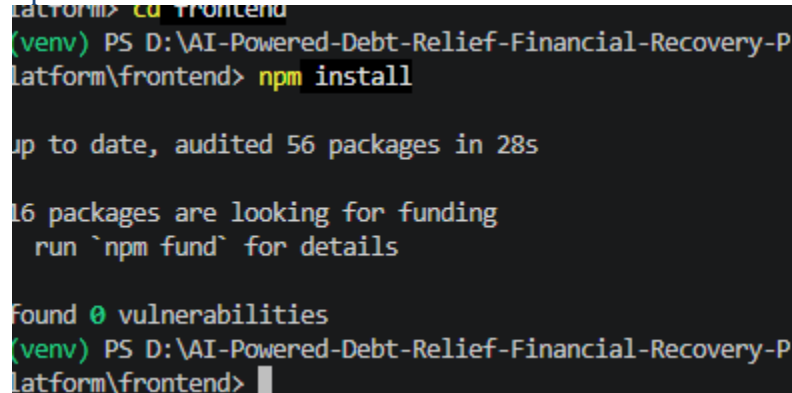
The frontend application was developed using React.js with Vite to provide a fast, responsive, and modern user interface for borrowers.

Steps Performed

Created the React project using Vite.

Installed all frontend dependencies:

`npm install`



```
Platform> cd frontend
(venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\frontend> npm install

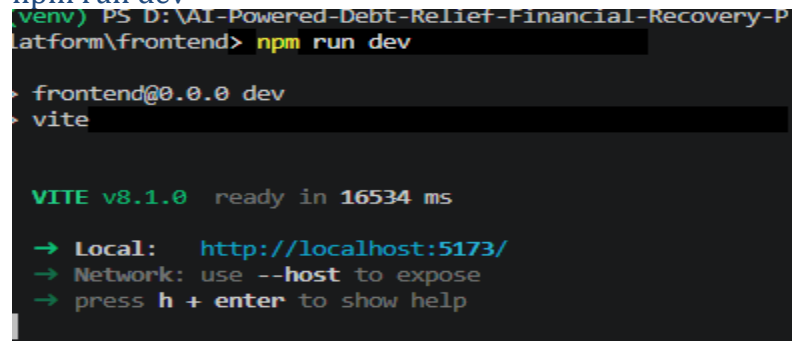
up to date, audited 56 packages in 28s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
(venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\frontend>
```

Started the frontend development server:

`npm run dev`



```
(venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\frontend> npm run dev

> frontend@0.0.0 dev
> vite

VITE v8.1.0 ready in 16534 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Verified successful rendering of the Financial Dashboard.

Story 4: Project Structure & Full-Stack Directory Organization

The project was organized using a modular architecture to separate frontend, backend, AI modules, database models, API routes, and utility components. This structure improves maintainability, scalability, and future enhancements.

Steps Performed

Created separate folders for frontend and backend.
 Organized backend modules into:

- API Routes
- Database Models
- Services
- AI Engine
- Financial Engine

```
latform\frontend> dir

Directory: D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\frontend

Mode                LastWriteTime         Length Name
----                -
d-----        6/30/2026   4:24 PM          dist
d-----        7/3/2026   12:17 PM        node_modules
d-----        6/26/2026   11:41 PM          public
d-----        6/30/2026   11:59 AM          src
-a-----        6/26/2026   11:36 PM          253 .gitignore
-a-----        6/26/2026   11:36 PM          231 .oxlint.json
-a-----        6/26/2026   11:41 PM          360 index.html
-a-----        7/3/2026    2:03 PM       57158 package-lock.js
-a-----        7/3/2026   12:17 PM          518 package.json
-a-----        6/26/2026   11:36 PM       1009 README.md
-a-----        6/26/2026   11:36 PM          161 vite.config.js
```

Organized frontend into reusable React components and pages.

```
latform> cd backend
(venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform>
latform\backend> dir

Directory: D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\backend

Mode                LastWriteTime         Length Name
----                -
d-----        7/3/2026   11:31 AM          app
d-----        6/28/2026    1:01 PM          venv
-a-----        6/27/2026    9:33 PM          184 .env
-a-----        6/27/2026    9:57 PM           4 .gitignore
-a-----        6/28/2026   10:47 PM          141 dependencies.py
-a-----        6/28/2026    9:26 PM       32768 finrelief.db
-a-----        7/3/2026    2:02 PM           86 package-lock.js
-a-----        6/27/2026    4:21 PM          126 requirements.txt

(venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\backend>
```

Verified communication between frontend and backend module.

Epic 2: AI Integration & Financial Processing Setup

It focuses on implementing the core business logic and AI capabilities of the FinRelief AI platform. During this phase, REST APIs were developed using FastAPI, financial analysis modules were created to evaluate borrower financial health, settlement prediction algorithms were implemented, and Google Gemini AI was integrated to generate intelligent negotiation strategies and settlement letters. A rule-based fallback mechanism was also developed to ensure uninterrupted functionality when the AI service is unavailable.

Story 1: FastAPI Backend & API Endpoints

A high-performance REST API backend was developed using FastAPI to handle communication between the React frontend and backend services. The APIs process borrower information, loan details, financial analysis, settlement prediction, and AI-generated negotiation strategies.

Steps Performed:

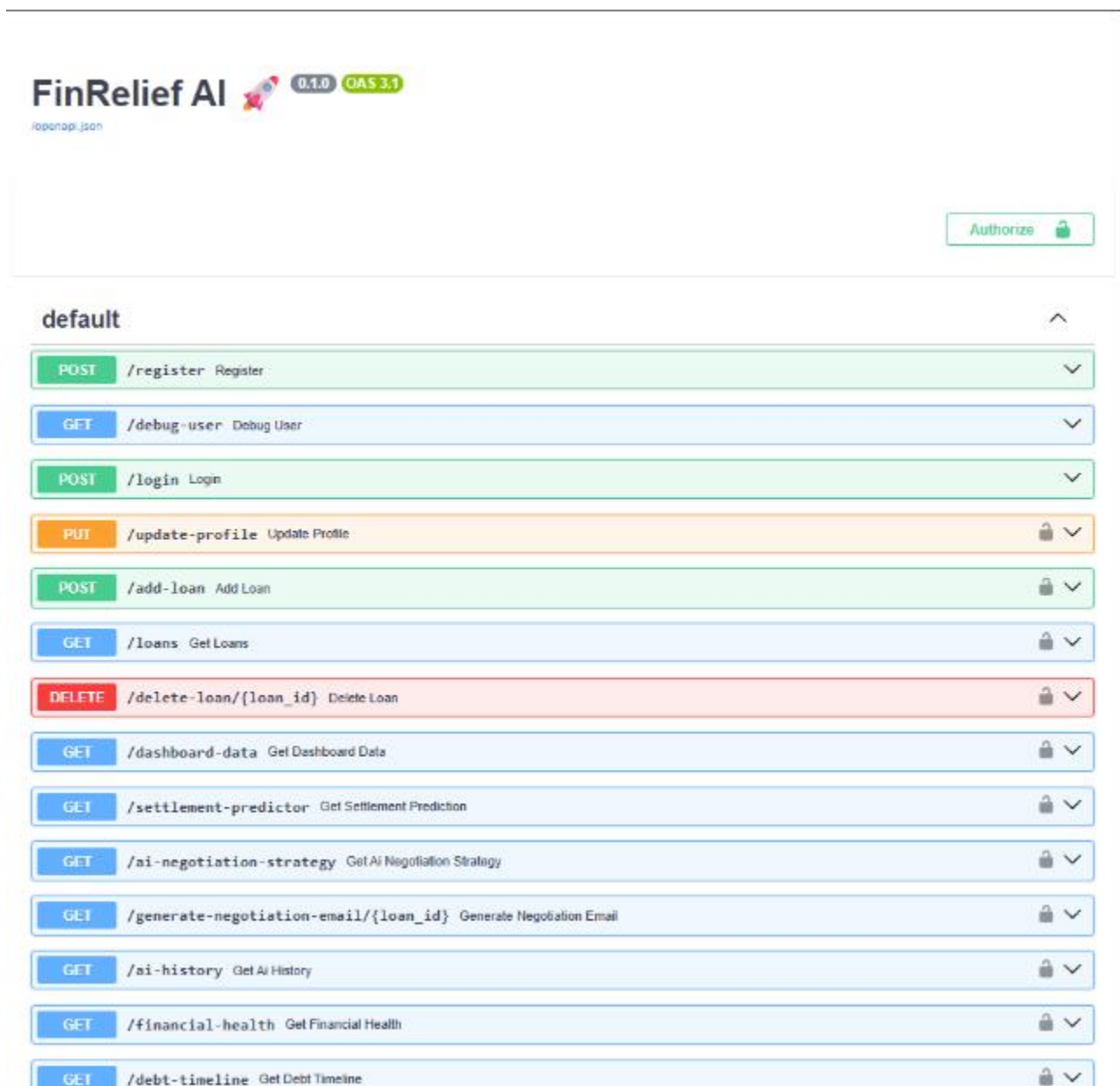
Installed and configured FastAPI and Uvicorn.

```
venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-P
atform> pip install fastapi uvicorn
Requirement already satisfied: fastapi in .\venv\Lib\sit
-packages (0.138.1)
Requirement already satisfied: uvicorn in .\venv\Lib\sit
-packages (0.49.0)
Requirement already satisfied: starlette>=0.46.0 in .\ve
nv\Lib\site-packages (from fastapi) (1.3.1)
Requirement already satisfied: pydantic>=2.9.0 in .\venv
\Lib\site-packages (from fastapi) (2.13.4)
Requirement already satisfied: typing-extensions>=4.8.0
 in .\venv\Lib\site-packages (from fastapi) (4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2
 in .\venv\Lib\site-packages (from fastapi) (0.4.2)
Requirement already satisfied: annotated-doc>=0.0.2 in .
\venv\Lib\site-packages (from fastapi) (0.0.4)
Requirement already satisfied: click>=7.0 in .\venv\Lib\
site-packages (from uvicorn) (8.4.2)
Requirement already satisfied: h11>=0.8 in .\venv\Lib\si
e-packages (from uvicorn) (0.16.0)
Requirement already satisfied: colorama in .\venv\Lib\si
e-packages (from click>=7.0->uvicorn) (0.4.6)
Requirement already satisfied: annotated-types>=0.6.0 in
.\venv\Lib\site-packages (from pydantic>=2.9.0->fastapi
(0.7.0)
Requirement already satisfied: pydantic-core==2.46.4 in
.\venv\Lib\site-packages (from pydantic>=2.9.0->fastapi)
(2.46.4)
Requirement already satisfied: anyio<5,>=3.6.2 in .\venv
Lib\site-packages (from starlette>=0.46.0->fastapi) (4.
4.1)
Requirement already satisfied: idna>=2.8 in .\venv\Lib\s
ite-packages (from anyio<5,>=3.6.2->starlette>=0.46.0->f
astapi) (3.18)
venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-P
atform>
```

shadabkhanam26 (4 days ago)

Created REST API endpoints for:

- User Authentication
 - Loan Management
 - Financial Health Analysis
 - Settlement Prediction
 - AI Negotiation Strategy Generation
- Implemented request validation using Pydantic models.
 - Tested APIs using FastAPI Swagger UI and Postman.



Verified JSON responses for all endpoints.

Story 2: Financial Engine Module

The Financial Engine was developed to calculate the borrower's financial condition based on income, expenses, EMIs, and outstanding loans. The system automatically evaluates financial health and classifies debt stress levels.

Steps Performed:

Function: EMI & Debt-to-Income Ratio Calculation

Calculates EMI ratio and debt-to-income percentage using borrower monthly income, expenses, and total loan obligations for financial health analysis.

```
def calculate_financial_health(user, loans):  
    """Calculate financial health metrics"""  
    total_emi = sum(loan.emi for loan in loans)  
    total_outstanding = sum(loan.outstanding_amount for loan in loans)  
    surplus = user.monthly_income - user.monthly_expenses - total_emi  
  
    # EMI Ratio = (total_emi / monthly_income) * 100  
    if user.monthly_income > 0:  
        emi_ratio = (total_emi / user.monthly_income) * 100  
    else:  
        emi_ratio = 0  
  
    # Debt-to-Income = (total_outstanding / monthly_income) * 100  
    if user.monthly_income > 0:  
        debt_to_income = (total_outstanding / user.monthly_income) * 100  
    else:  
        debt_to_income = 0
```

Function: Financial Stress Level Classification

Classifies borrower debt stress into Low, Medium, and High categories based on calculated EMI ratio and repayment burden analysis.

```
def calculate_financial_health(user, loans):  
    # Stress Level Logic:  
    # <30 → Low  
    # 30-50 → Medium  
    # >50 → High  
    if emi_ratio > 50:  
        stress_level = "High"  
    elif emi_ratio >= 30:  
        stress_level = "Medium"  
    else:  
        stress_level = "Low"  
  
    return {  
        "total_emi": total_emi,  
        "total_outstanding": total_outstanding,  
        "surplus": surplus,  
        "emi_ratio_percent": round(emi_ratio, 2),  
        "debt_to_income_percent": round(debt_to_income, 2),  
        "stress_level": stress_level,  
        "total_loans": len(loans)  
    }
```

Loan Priority Analysis

Analyzes borrower loan portfolio and assigns priority levels based on overdue duration, interest rate, and EMI burden for intelligent debt management.

```
def calculate_loan_priority(loans, emi_ratio=0):  
    priority_list = []  
  
    for loan in loans:  
        # Determine priority based on rules  
        is_overdue = loan.overdue_months > 0  
        high_interest = loan.interest_rate > 12  
        high_emi_ratio = emi_ratio > 50  
  
        if is_overdue or high_interest or high_emi_ratio:  
            priority = "High"  
        elif loan.interest_rate > 8 or loan.overdue_months > 0:  
            priority = "Medium"  
        else:  
            priority = "Low"  
  
        priority_list.append({  
            "loan_id": loan.id,  
            "lender_name": loan.lender_name,  
            "outstanding_amount": loan.outstanding_amount,  
            "interest_rate": loan.interest_rate,  
            "overdue_months": loan.overdue_months,  
            "emi": loan.emi,  
            "priority": priority  
        })
```

Function: Loan Priority Analysis

Analyzes borrower loan portfolio and assigns priority levels based on overdue duration, interest rate, and EMI burden for intelligent debt management.

```
# Sort by priority (High first)  
priority_order = {"High": 0, "Medium": 1, "Low": 2}  
priority_list.sort(key=lambda x: priority_order[x["priority"]])  
  
return priority_list  
  
def simulate_debt_timeline(user, loans, extra_payment=0):  
    loan_data = [  
        {  
            "loan_id": loan.id,  
            "balance": loan.outstanding_amount,  
            "interest_rate": loan.interest_rate,  
            "emi": loan.emi  
        }  
        for loan in loans  
    ]  
  
    months = 0  
    max_months = 240  
    timeline = []  
  
    while any(loan["balance"] > 0 for loan in loan_data) and months < max_months:  
        months += 1  
        total_balance = 0  
  
        loan_data.sort(key=lambda x: x["balance"], reverse=True)
```

Function: Priority-Based Loan Sorting

Sorts loan records according to financial urgency and repayment priority to improve settlement planning and negotiation strategy generation.

```
for loan in loan_data:
    if loan["balance"] <= 0:
        continue

    monthly_interest = (loan["interest_rate"] / 100) / 12
    loan["balance"] += loan["balance"] * monthly_interest

    payment = loan["emi"]

    if extra_payment > 0 and loan == loan_data[0]:
        payment += extra_payment

    loan["balance"] -= payment

    if loan["balance"] < 0:
        loan["balance"] = 0

    total_balance += loan["balance"]

    timeline.append({
        "month": months,
        "remaining_balance": round(total_balance, 2)
    })
```

Function: Debt Repayment Timeline Simulation

Simulates monthly debt repayment progression and balance reduction scenarios based on borrower repayment capacity and additional payment strategies.

Verified calculations using sample financial data.

```
    })

    return {
        "months_to_debt_free": months,
        "final_remaining_balance": round(total_balance, 2),
        "timeline_preview": timeline[:12]
    }
```

Story 3: Settlement Prediction System

A settlement prediction module was developed to recommend optimal settlement amounts by analyzing borrower financial information and loan characteristics.

Steps Performed:

Collected borrower loan information.

Analyzed:

- Outstanding Amount
- Overdue Duration
- Loan Type
- Financial Health

```
# Calculate Debt-to-Income ratio
if user.monthly_income > 0:
    debt_to_income = (total_outstanding / user.monthly_income) * 100
else:
    debt_to_income = 0

settlement_results = []

for loan in loans:
    # Base settlement = 50%
    settlement = 50.0
    risk_score = 0

    # +5% if overdue
    if loan.overdue_months > 0:
        settlement += 5
        risk_score += 20

    # +5% if EMI ratio > 50
    if emi_ratio > 50:
        settlement += 5
        risk_score += 15
```



```
# +5% if interest_rate > 12%
if loan.interest_rate > 12:
    settlement += 5
    risk_score += 10

# +5% if debt_to_income > 80%
if debt_to_income > 80:
    settlement += 5
    risk_score += 15

# Clamp settlement: Minimum 40%, Maximum 75%
settlement = max(40, min(75, settlement))

# Determine risk category
if risk_score >= 40:
    risk_category = "High"
elif risk_score >= 20:
    risk_category = "Medium"
else:
    risk_category = "Low"

settlement_results.append({
    "loan_id": loan.id,
    "lender_name": loan.lender_name,
    "outstanding_amount": loan.outstanding_amount,
    "interest_rate": loan.interest_rate,
    "emi": loan.emi,
    "suggested_settlement_percentage": settlement,
    "risk_score": risk_score,
    "risk_category": risk_category
})

return settlement_results
```

Calculated recommended settlement percentage. Assigned settlement priority (High, Medium, Low). Generated structured settlement recommendations. Returned prediction results through FastAPI APIs.

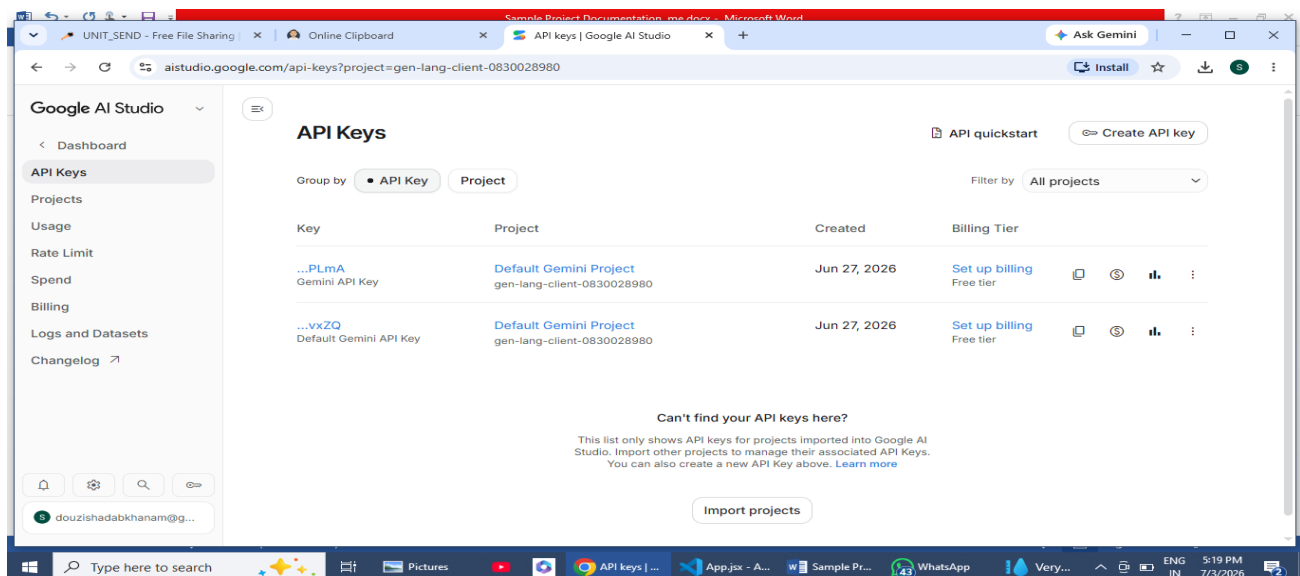
Story 4: AI Negotiation Strategy Engine

Google Gemini AI was integrated into the platform to automatically generate professional negotiation strategies and settlement request letters based on borrower financial conditions.

Steps Performed

- Integrated Google Gemini API.
- Created AI prompt templates.
- Sent borrower financial profile to Gemini.

Generated:



- Negotiation Strategy, Settlement Recommendation, Professional Negotiation Letter

```
import os
import json
from dotenv import load_dotenv

load_dotenv()

GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY", "")

def _call_gemini(prompt: str) -> str:
    """Call Google Gemini API if key is available, otherwise use rule-based fallback."""
    if not GOOGLE_API_KEY:
        return None # Will fall through to fallback

    try:
        import google.generativeai as genai
        genai.configure(api_key=GOOGLE_API_KEY)
        model = genai.GenerativeModel("gemini-1.5-flash")
        response = model.generate_content(prompt)
        return response.text
    except ImportError:
        return None
    except Exception as e:
        print(f"Gemini API error: {e}")
        return None
```

Displayed AI responses on the frontend.

Stored generated AI history for future reference.

Story 5: Fallback Logic Implementation

A rule-based fallback engine was developed to ensure continuous operation whenever the Google Gemini API is unavailable or an API error occurs.

Steps Performed:

Implemented try-except error handling. Detected Gemini API failures. Created predefined negotiation templates.

- Returned rule-based recommendations during failures.
- Prevented backend crashes.
- Ensured uninterrupted borrower experience.

```
GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY", "")

def _call_gemini(prompt: str) -> str:
    """Call Google Gemini API if key is available, otherwise use rule-based fallback."""
    if not GOOGLE_API_KEY:
        return None # Will fall through to fallback

    try:
        genai = importlib.import_module("google.generativeai")
        genai.configure(api_key=GOOGLE_API_KEY)
        model = genai.GenerativeModel("gemini-1.5-flash")
        response = model.generate_content(prompt)
        return response.text
    except ImportError:
        return None
    except Exception as e:
        print(f"Gemini API error: {e}")
        return None
```

Epic 3: Database Management & Financial Data Storage Setup

It focuses on designing, implementing, and managing the database layer of the FinRelief AI – AI Powered Debt Relief & Financial Recovery Platform. During this phase, the backend was integrated with a SQLite database using SQLAlchemy ORM to securely store borrower information, loan records, financial profiles, settlement predictions, and AI-generated negotiation history. APIs were developed for data storage, retrieval, and processing while ensuring secure authentication and efficient database operations.

Story 1: API Development Functionality

FastAPI APIs were developed to process borrower information, validate input data, communicate with the database, and return structured responses to the frontend.

Steps Performed:

- Created REST API endpoints for borrower operations.
- Validated request data using Pydantic schemas.
- Integrated SQLAlchemy ORM with FastAPI.
- Connected APIs with SQLite database.
- Processed frontend requests and returned JSON responses.

```
@router.get("/ai-negotiation-strategy")
def get_ai_negotiation_strategy(db: Session = Depends(get_db), current_user: User = Depends(get_current_user)):
    try:
        user = db.query(User).filter(User.id == current_user.id).first()
        loans = db.query(Loan).filter(Loan.user_id == user.id).all()
        if not loans:
            return {"strategy": "Please add at least one loan to generate an AI strategy."}
        financial_health = calculate_financial_health(user, loans)
        settlement_data = calculate_settlement_probability(user, loans)
        strategy = generate_negotiation_strategy(user, loans, financial_health, settlement_data)
        try:
            db.add(AIHistory(user_id=user.id, query_type="Negotiation Strategy", response=strategy))
            db.commit()
        except Exception:
            db.rollback()
        return {"strategy": strategy}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"AI Strategy error: {str(e)}")
```

Verified API functionality using FastAPI Swagger UI and Postman.

Story 2: Loan & Settlement Processing Functionality

Loan management and settlement processing modules were developed to store loan records, calculate settlement recommendations, assign priority levels, and generate negotiation-related information.

Steps Performed:

- Stored borrower loan details in the database.
- Retrieved loan information using SQLAlchemy queries.
- Calculated settlement percentage based on loan details.
- Assigned High, Medium, or Low settlement priority.
- Generated settlement recommendation records.
- Stored AI-generated negotiation letters and settlement history.

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
import os

from dotenv import load_dotenv

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./finrelief.db")

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False},
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()
```

Verified successful database transactions.

Story 3: Data Handling

The platform securely manages borrower data, loan information, financial profiles, settlement records, and AI interaction history using SQLite and SQLAlchemy ORM while maintaining efficient retrieval and authentication mechanisms.

Steps Performed:

- Created database models using SQLAlchemy ORM.
- Implemented CRUD (Create, Read, Update, Delete) operations.
- Stored user, loan, and settlement information securely.
- Maintained AI-generated negotiation history.
- Enabled efficient database queries for borrower records, Protected API access using JWT authentication, Tested data storage and retrieval successfully.

```
from sqlalchemy import Column, Integer, Float, String
from app.database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)

    # 🛡️ Authentication Fields
    email = Column(String(255), unique=True, index=True, nullable=False)
    password = Column(String(255), nullable=False)

    # 💰 Financial Profile
    monthly_income = Column(Float, nullable=False, default=0.0)
    monthly_expenses = Column(Float, nullable=False, default=0.0)
    lump_sum_available = Column(Float, nullable=False, default=0.0)
```

Epic 4: Frontend Integration & User Interface Development

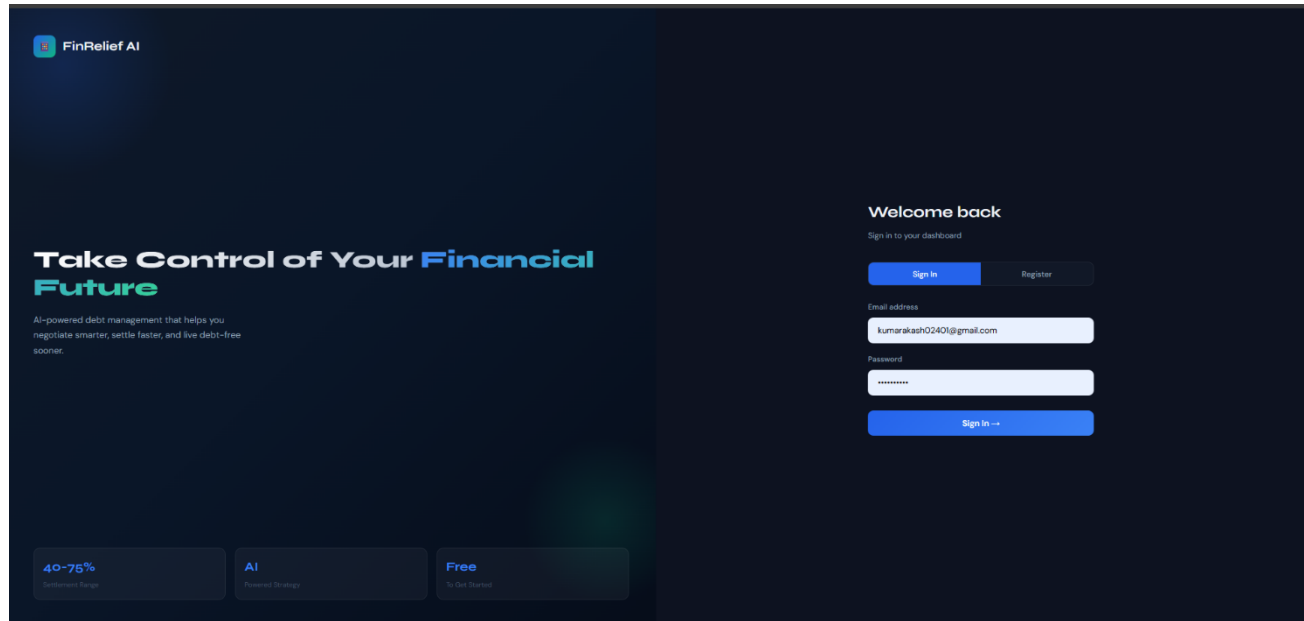
It focuses on developing and integrating the frontend user interface of the FinRelief AI – AI Powered Debt Relief & Financial Recovery Platform. During this phase, a responsive React.js interface was developed, integrated with FastAPI backend APIs, and enhanced with financial dashboard visualization, AI-generated settlement recommendations, and borrower-friendly UI components. The frontend provides an interactive environment for borrowers to analyze their financial health and access AI-powered debt recovery services.

Story 1: User Interface Development

A modern and responsive user interface was developed using React.js to provide borrowers with a clean and interactive financial dashboard for managing loans, viewing financial insights, and accessing AI-powered features.

Steps Performed:

- Designed the frontend using React.js.
- Created reusable UI components, Developed Financial Dashboard interface.
- Displayed borrower financial metrics.
- Implemented responsive layout for desktop and mobile devices.
- Applied professional styling using CSS.



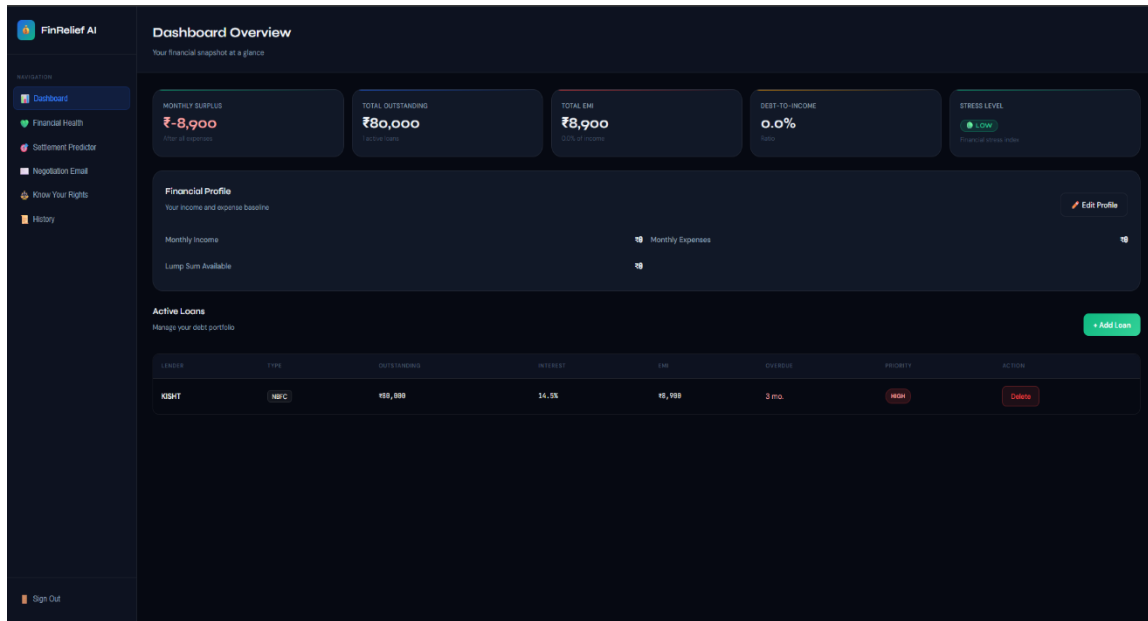
Verified successful frontend rendering.

Story 2: Frontend Communication with FastAPI

The React frontend was integrated with FastAPI backend services to retrieve financial data, settlement recommendations, and AI-generated negotiation responses through secure REST API communication.

Steps Performed:

- Configured Axios for API communication, Connected React frontend with FastAPI backend.
- Sent borrower financial data to backend APIs.
- Retrieved settlement prediction results.
- Displayed AI-generated negotiation responses.
- Implemented JWT token handling for secure API requests.



Verified successful frontend-backend communication.

Story 3: Financial Health Metrics & Settlement Data Visualization

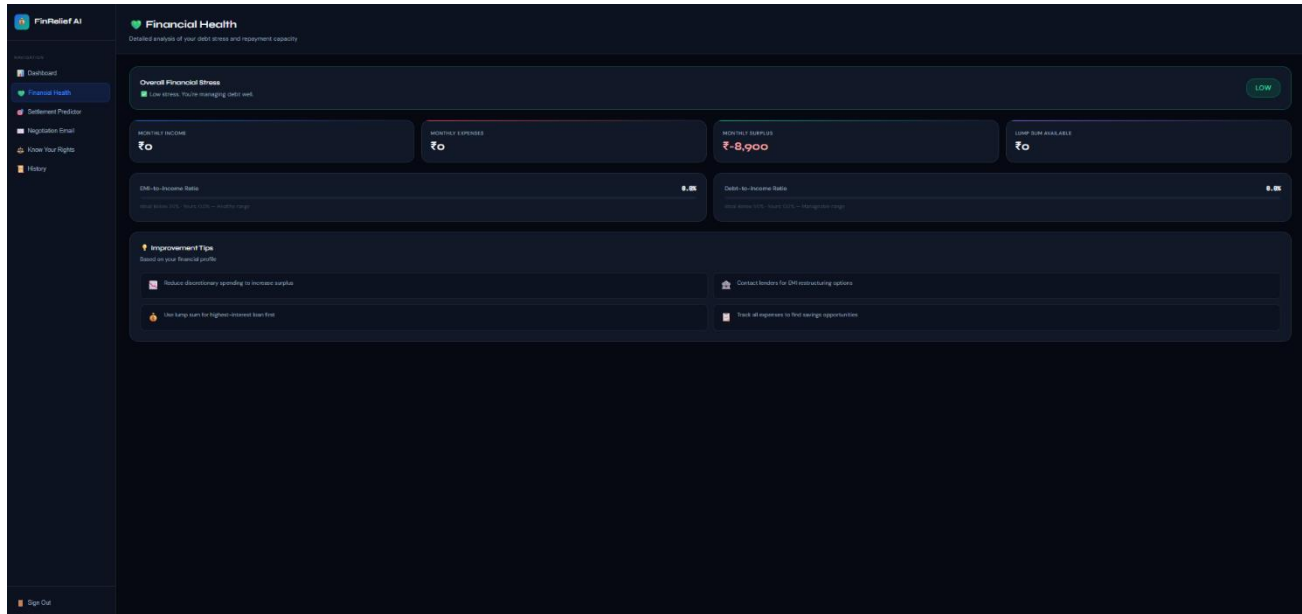
An interactive financial dashboard was developed to display borrower financial health using real-time analytics and visual indicators for better financial decision-making.

Steps Performed:

- Displayed Monthly Surplus, Displayed Total Outstanding Amount.
- Displayed EMI-to-Income Ratio.
- Displayed Debt Stress Level.
- Displayed Settlement Recommendation Percentage.
- Implemented financial summary cards.
- Updated dashboard dynamically using backend API responses.

Financial Dashboard Interface

Displays borrower financial overview including active loans, monthly surplus, debt stress analysis.



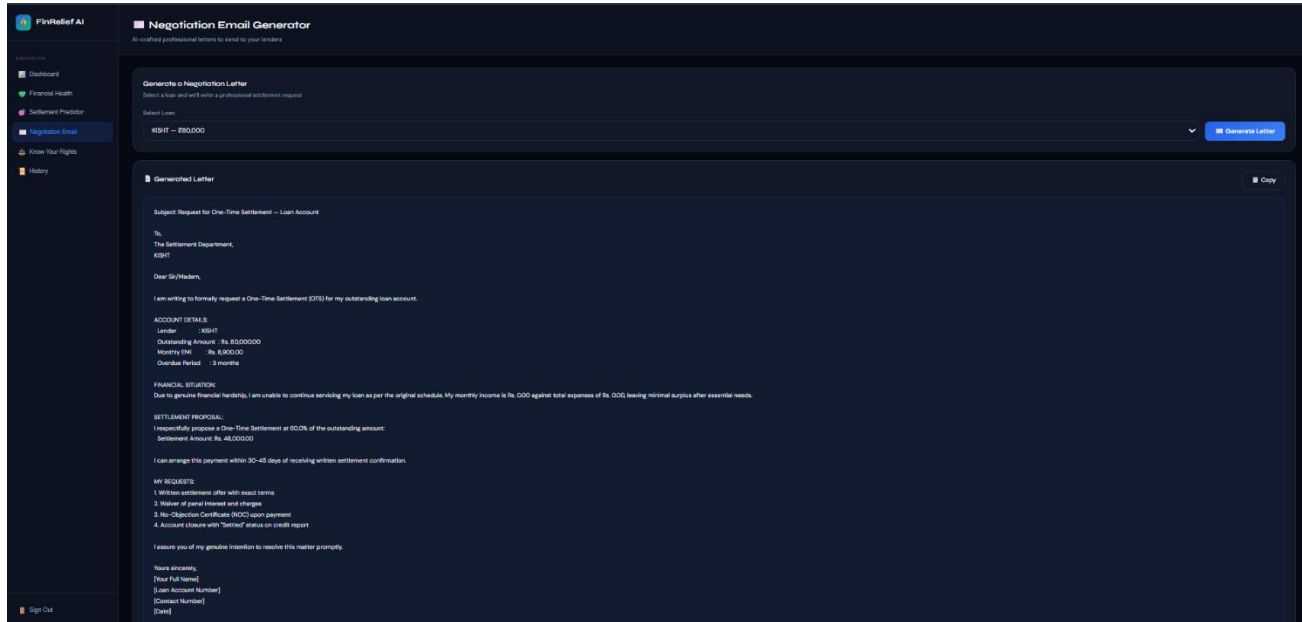
AI-powered Settlement Prediction

Generates intelligent settlement recommendations and repayment analysis based on borrower financial profile.



AI Negotiation Strategy Generation

Creates lender-specific AI-Powered negotiation letters and settlements request content using Google Gemini API integration.



Verified visualization accuracy.

Story 4: UI Enhancements

The user interface was enhanced with responsive layouts, modern styling, improved accessibility, and smooth user interaction to provide a professional borrower experience.

Steps Performed:

- Improved dashboard layout.
- Enhanced responsive design.
- Optimized CSS styling.
- Added interactive dashboard cards.
- Improved typography and spacing.
- Enhanced frontend performance.
- Ensured consistent UI theme across the application.

```

:root {
  --bg-base: #060912;
  --bg-surface: #0d1220;
  --bg-card: #111827;
  --bg-hover: #1a2235;
  --border: rgba(255,255,255,0.08);
  --border-bright: rgba(255,255,255,0.15);

  --accent-blue: #2563eb;
  --accent-blue-light: #3b82f6;
  --accent-green: #10b981;
  --accent-green-light: #34d399;
  --accent-red: #ef4444;
  --accent-amber: #f59e0b;
  --accent-purple: #8b5cf6;

  --text-primary: #f1f5f9;
  --text-secondary: #94a3b8;
  --text-muted: #475569;

  --font-display: 'Syne', sans-serif;
  --font-body: 'DM Sans', sans-serif;
  --font-mono: 'JetBrains Mono', monospace;

  --radius: 12px;
  --radius-lg: 20px;
  --radius-xl: 28px;
  --sidebar-width: 260px;

```

Verified smooth user interaction.

Epic 5: Testing, Debugging & Performance Optimization

It focuses on validating the functionality, reliability, security, and performance of the FinRelief AI – AI Powered Debt Relief & Financial Recovery Platform. During this phase, the application was tested under different financial scenarios, backend errors were handled effectively, AI fallback mechanisms were verified, and performance optimizations were implemented to ensure stable and secure execution of the platform.

Story 1: System Testing

Comprehensive testing was performed to verify the functionality of the frontend, backend APIs, financial engine, settlement prediction module, and AI-powered negotiation system under different borrower scenarios.

Steps Performed:

System testing was carried out by executing the complete FinRelief AI platform under different borrower financial scenarios to validate overall functionality. The frontend and backend integration was verified using FastAPI APIs, while financial health calculations, settlement prediction modules, and AI-powered negotiation strategy generation were tested for accuracy. API responses were validated through FastAPI Swagger UI and Postman to ensure correct communication between the frontend and backend. Different user inputs and loan conditions were tested to confirm reliable system behavior and smooth application performance.

```
@router.put("/update-profile")
def update_profile(profile: ProfileUpdate, db: Session = Depends(get_db), current_user: User = Depends(get_current_user)):
    try:
        user = db.query(User).filter(User.id == current_user.id).first()
        user.monthly_income = profile.monthly_income
        user.monthly_expenses = profile.monthly_expenses
        user.lump_sum_available = profile.lump_sum_available
        db.commit()
        return {"message": "Profile Updated Successfully"}
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=f"Failed to update profile: {str(e)}")
```

```
@router.post("/add-loan")
def add_loan(loan: LoanCreate, db: Session = Depends(get_db), current_user: User = Depends(get_current_user)):
    try:
        new_loan = Loan(
            user_id=current_user.id, lender_name=loan.lender_name,
            outstanding_amount=loan.outstanding_amount, interest_rate=loan.interest_rate,
            emi=loan.emi, overdue_months=loan.overdue_months, loan_type=loan.loan_type
        )
        db.add(new_loan)
        db.commit()
        db.refresh(new_loan)
        return {"message": "Loan Added Successfully", "id": new_loan.id}
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=f"Failed to add loan: {str(e)}")
```

Verified overall system stability and functionality.

Story 2: Backend Error Handling & AI Fallback Management

Robust exception handling and fallback mechanisms were implemented to ensure uninterrupted platform operation during API failures, invalid inputs, and unexpected backend errors.

Steps Performed:

Backend error handling was implemented by incorporating structured exception handling using try-except blocks across all critical backend modules. User inputs were validated using Pydantic schemas to prevent invalid requests from reaching the application. During Google Gemini API failures or missing API key scenarios, the system automatically activated the rule-based fallback engine to generate predefined negotiation strategies and settlement responses. User-friendly error messages were displayed on the frontend while ensuring that backend services remained stable without application crashes.

```
def _call_gemini(prompt: str) -> str:
    """Call Google Gemini API if key is available, otherwise use rule-based fallback."""
    if not GOOGLE_API_KEY:
        return None # Will fall through to fallback

    try:
        genai = importlib.import_module("google.generativeai")
        genai.configure(api_key=GOOGLE_API_KEY)
        model = genai.GenerativeModel("gemini-1.5-flash")
        response = model.generate_content(prompt)
        return response.text
    except ImportError:
        return None
    except Exception as e:
        print(f"Gemini API error: {e}")
        return None
```

Verified backend stability under failure scenarios.

Story 3: Performance Optimization & Secure Session Management

Performance optimization techniques were applied to improve application responsiveness, database efficiency, authentication security, and overall system performance.

Steps Performed:

Performance optimization focused on improving application responsiveness, authentication security, and database efficiency. JWT authentication was configured to provide secure user sessions, while SQLite database operations and SQLAlchemy queries were optimized for faster data retrieval. Axios interceptors were implemented to securely attach authentication tokens to API requests and prevent duplicate requests. Password hashing using Werkzeug improved authentication security, and frontend-backend communication was optimized to reduce response time, ensuring stable and efficient platform performance under different testing conditions.

```
@router.post("/login")
def login(
    form_data: OAuth2PasswordRequestForm = Depends(),
    db: Session = Depends(get_db),
):
    try:
        # OAuth2PasswordRequestForm uses "username" field to carry whatever the client sends.
        email = (form_data.username or "").strip().lower()
        password = form_data.password or ""

        user = db.query(User).filter(User.email == email).first()
        if not user:
            logger.warning("Login failed: user not found for email=%s", email)
            raise HTTPException(status_code=401, detail="Invalid email or password")

        if not verify_password(password, user.password):
            logger.warning("Login failed: password mismatch for email=%s", email)
            raise HTTPException(status_code=401, detail="Invalid email or password")

        token = create_token({"sub": user.email})
        return {"access_token": token, "token_type": "bearer"}
    except HTTPException:
        raise
    except Exception as e:
        logger.exception("Login failed with server error")
        raise HTTPException(status_code=500, detail=f"Login failed: {str(e)}")
```

Verified stable application performance during testing.

Epic 6: Version Control, Project Finalization & Deployment Readiness

It focuses on preparing the FinRelief AI – AI Powered Debt Relief & Financial Recovery Platform for project completion, source code management, and deployment readiness. During this phase, the complete project was organized into a modular folder structure, synchronized with a GitHub repository using Git version control, cleaned by removing unnecessary files, and configured for production deployment. Environment variables, backend server configuration, and frontend production builds were also prepared to ensure a secure, scalable, and deployment-ready application.

Story 1: GitHub Repository Setup & Version Control Management

The complete project source code was uploaded to GitHub, enabling version control, collaboration, and secure project backup throughout the development lifecycle.

Steps Performed :

- Installed and configured Git on the development system.
- Initialized the local Git repository using git init.
- Added project files using git add .
- Created meaningful commits using git commit.
- Connected the local repository to the GitHub remote repository.
- Pushed the complete source code using git push origin main.
- Verified successful synchronization with GitHub.
- Maintained version history for future project updates.


AI-Powered-Debt-Relief-Financial-Recovery-Platform
Public
Pin
Watch 0

main
1 Branch
0 Tags

Add file
Code

 shadabkhanam26 Epic 5 7253f30 · 3 days ago 14 Commits		
ER_Diagram	Initial project upload	last week
Pre-requisites	Initial project upload	last week
backend	Epic 5	3 days ago
frontend	Epic 4 Completed	3 days ago
README.md	Initial project upload	last week
package-lock.json	Initial project upload	last week


README

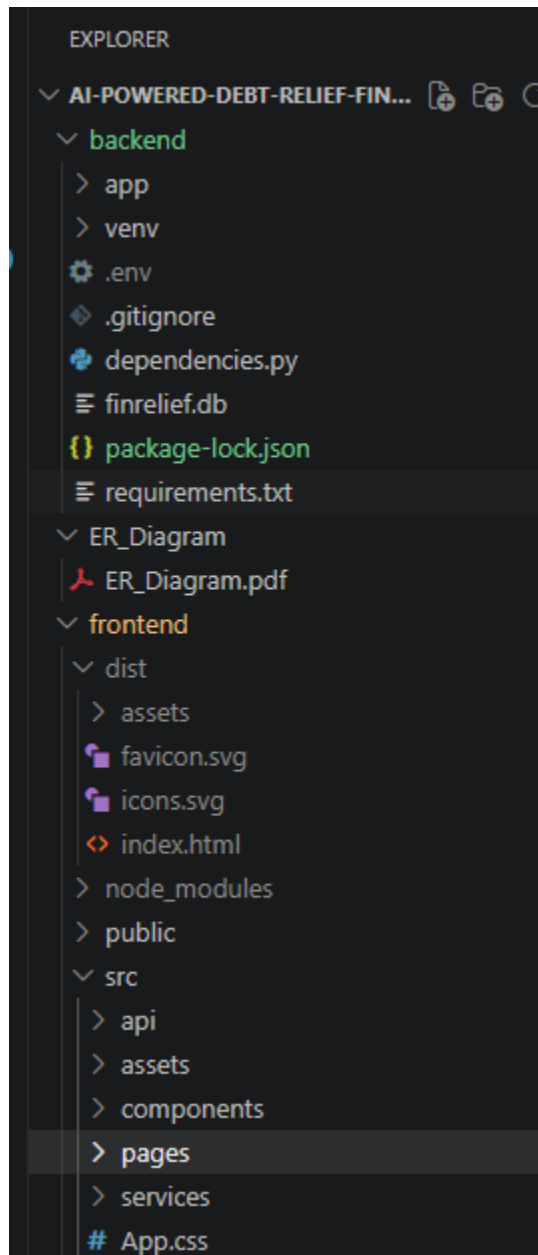

Story 2: Project Cleanup & Modular Folder Structure Organization

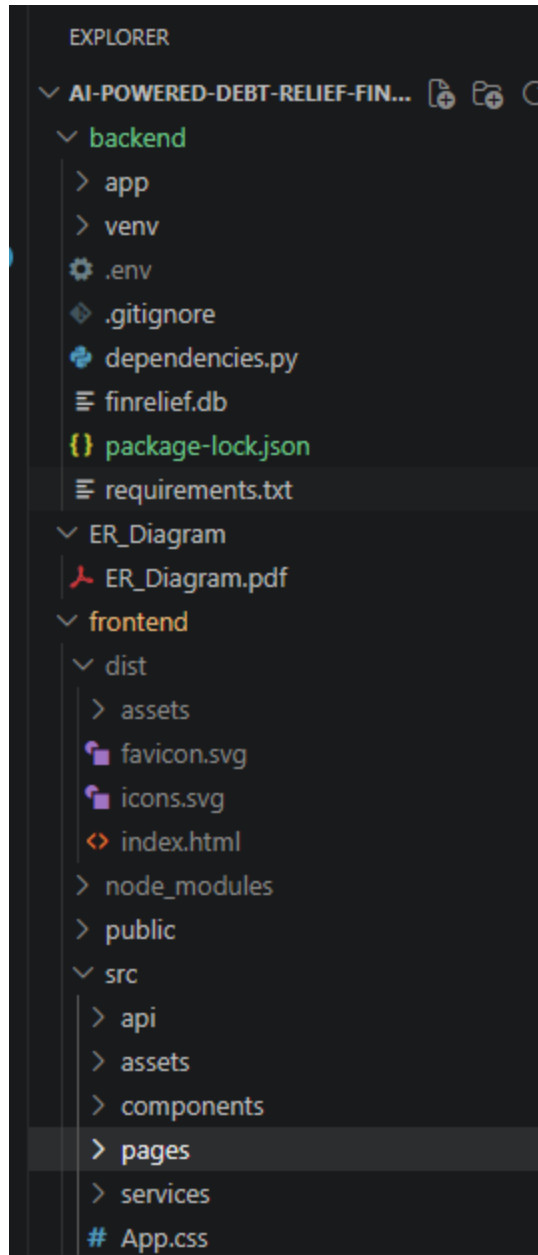
The project structure was optimized by organizing the application into reusable modules and removing unnecessary files to improve maintainability and readability.

Steps Performed :

- Removed unnecessary files and cache folders.
- Deleted duplicate and unused source files.
- Organized backend into modular folders (routes, models, services, utils).

- Organized frontend into reusable components and pages.
- Structured project directories for better maintainability.
- Improved code readability through modular organization.
- Verified project structure after cleanup.
- Prepared the project for future enhancements.





Story 3: Deployment Configuration & Production Readiness Setup

The application was configured for production deployment by preparing both frontend and backend environments and securing application configuration settings.

Steps Performed :

- Configured environment variables using the .env file.
- Prepared the React frontend for production using Vite build.
- Configured the FastAPI backend with the Uvicorn ASGI server.
- Verified frontend and backend communication.

- Secured API keys and authentication secrets.
- Validated production configuration settings.
- Tested application stability before deployment.
- Confirmed deployment readiness for cloud/server hosting.

```
(venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform>
    cd frontend
(venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\frontend> npm run build

> frontend@0.0.0 build
> vite build

vite v8.1.0 building client environment for production..
✓ 74 modules transformed.
computing gzip size...
dist/index.html                0.45 kB | gzip: 0.29
kB
dist/assets/index-DxccIns9.css  2.69 kB | gzip: 0.95
kB
dist/assets/index-BUPsIg7k.js 236.94 kB | gzip: 77.17
kB

[PLUGIN_TIMINGS] Your build spent significant time in pl
ugins. Here is a breakdown:
  - rolldown:vite-resolve (37%)
  - vite:build-html (29%)
  - vite:css-post (26%)
See https://rolldown.rs/reference/InputOptions.checks#pl
ugin timings for more details.

✓ built in 14.32s
(venv) PS D:\AI-Powered-Debt-Relief-Financial-Recovery-Platform\frontend>
```